

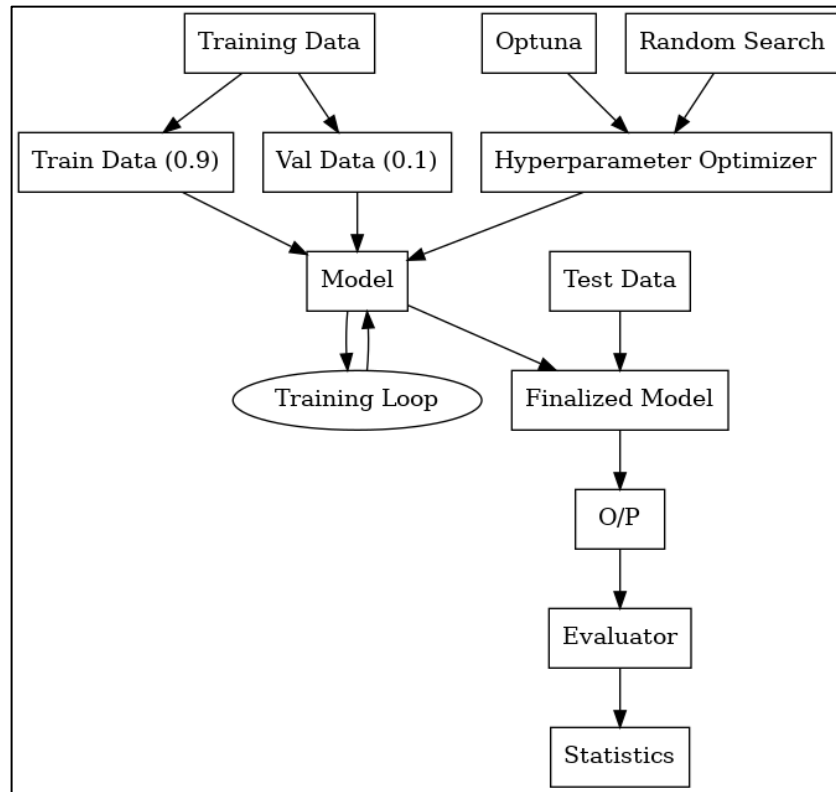
EEE 549: Statistical Machine Learning

Student: Gautham Damodaran Jagath Kumar

1229664841

Group 4

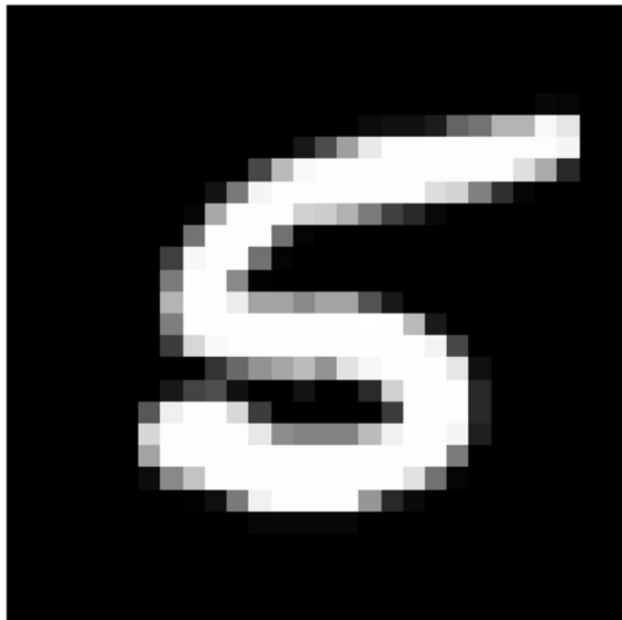
Training, Validation, and Testing Pipeline



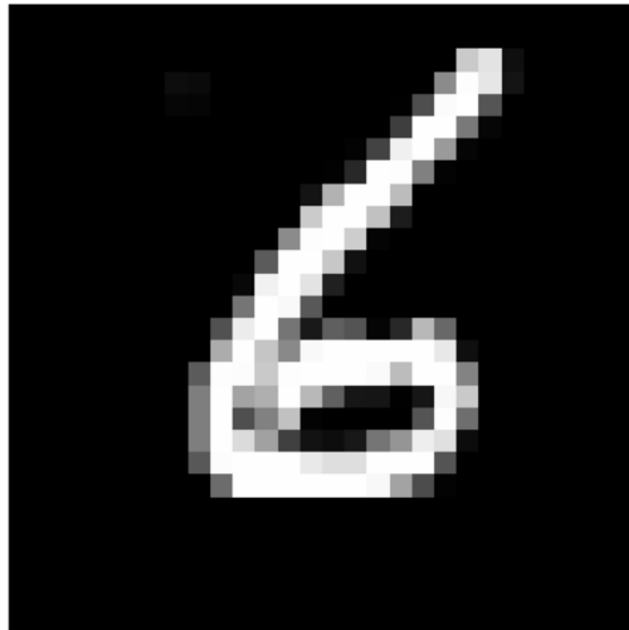
This diagram details the various components and states of the training pipeline

MNIST Dataset

Binary Classes



Digit 5



Digit 6

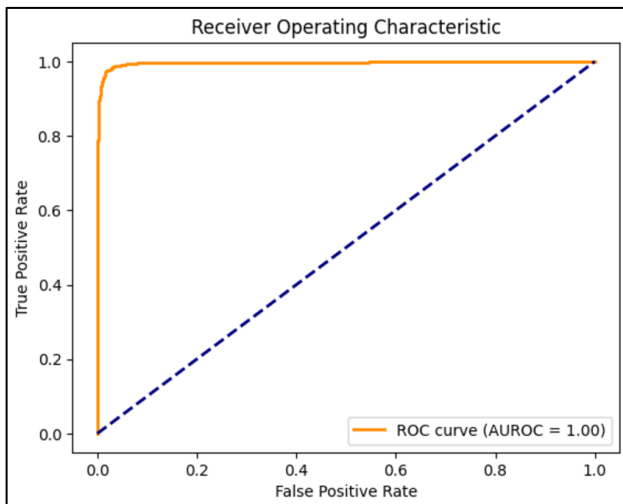
Logistic Regression

Logistic Regression Hyperparameters

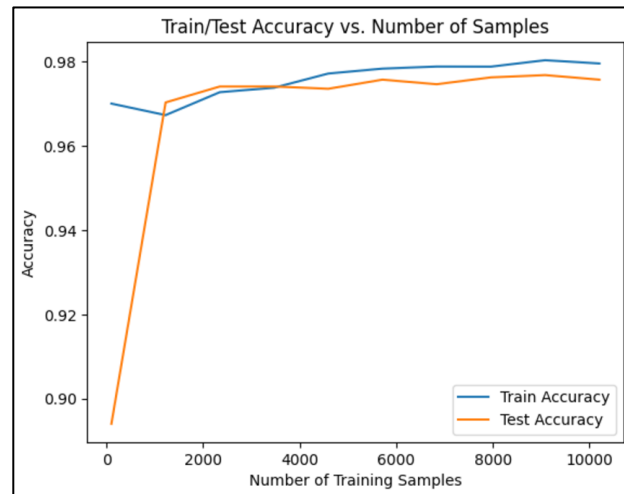
Parameter	Range	Description	Why Tuned?
C	{1e-4, 0.0005, 0.001, 0.0025, 0.005, 0.01, 0.1, 1, 10, 100, 1e+20}	The inverse of the regularization strength	Use to mitigate/ameliorate overfitting to the training data.
tol	{1e-3, 0.5*1e-3, 1e-4}	The precision level w.r.t which the model needs to converge	To decide at what point we can stop training the algorithm, balancing both precision and complexity
solver	{'lbfgs', 'sag', 'saga', 'liblinear', 'newton-cg', 'newton-cholesky'}	The different optimization algorithms used to minimize error	To find the optimization algorithm that best works with the given data

Results and Plots

Method	Accuracy	Per-class Accuracy (Digit 5/6)	AUROC
LR	97.57%	Digit 5: 0.9719730941704036 Digit 6: 0.9791231732776617	1



ROC Plot

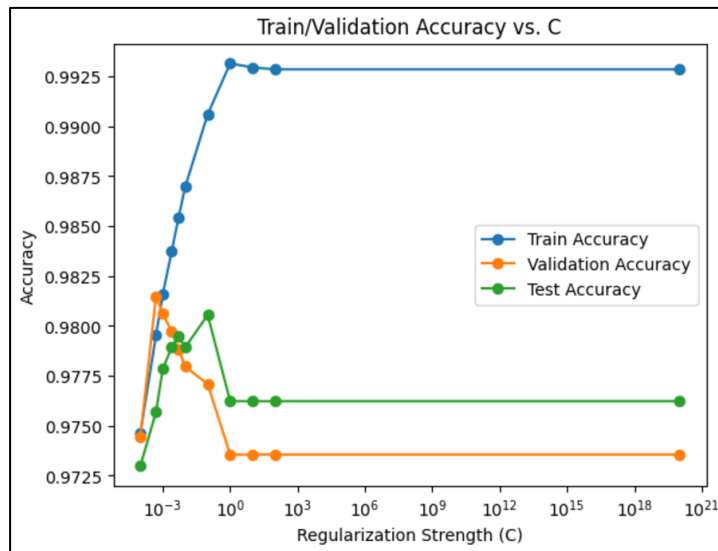


Sample Size Plot

Final Hyperparameters

LR Hyperparameters

Parameter	Value
C	0.0005
solver	lbfgs
tol	0.001
max_iter	2000
penalty	l2



We model the without regularization scenario using a very high C value ($1e+20$). Drawing graphs for two models on the same graph looked to be a bit weird and hence I opted to do this.

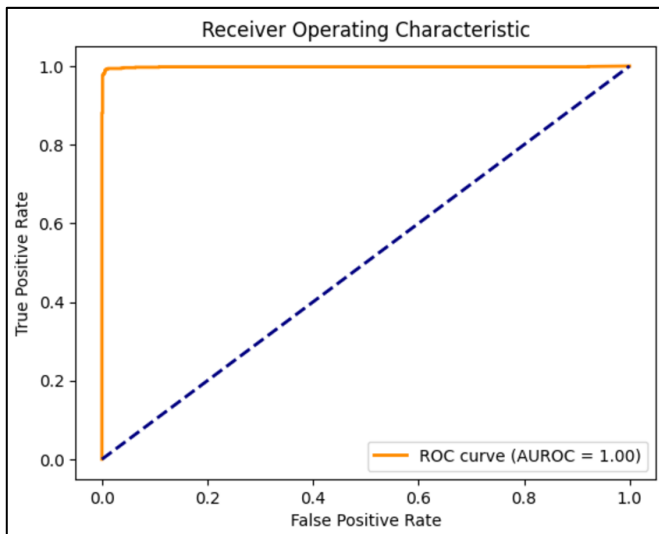
Support Vector Machines

SVM Hyperparameters

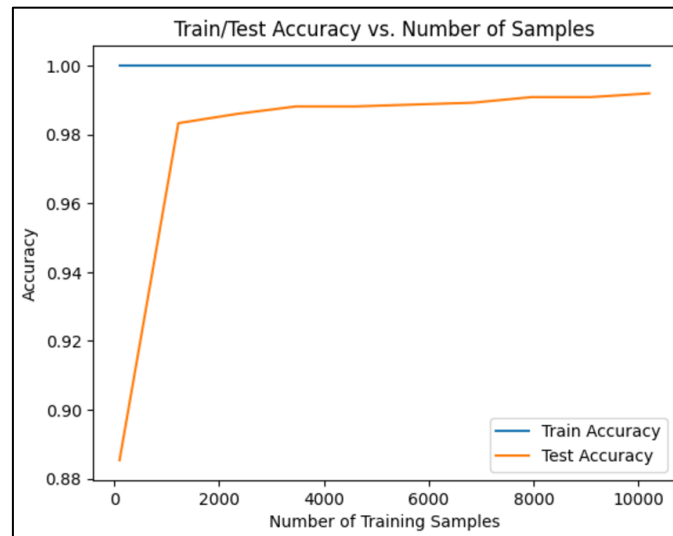
Parameter	Range	Description	Why Tuned?
C	{1e-4, 0.0005, 0.001, 0.0025, 0.005, 0.01, 0.1, 1, 10, 100}	Regularization Strength	Use to mitigate overfitting to the training data.
kernel	{'linear', 'poly', 'rbf', 'sigmoid'}	The kernel used to map the data to a different dimension	To experiment with different kernels and see what works best for the given data
tol	{1e-3, 0.5*1e-3, 1e-4}	The tolerance i.e precision to which the algorithm aims to converge	In order to balance efficiency and accuracy of the algorithm

Results and Plots

Method	Accuracy	Per-class Accuracy (+1/-1)	AUROC
SVM	99.19%	{5: 0.9910313901345291, 6: 0.9926931106471816}	1.0



ROC Plot

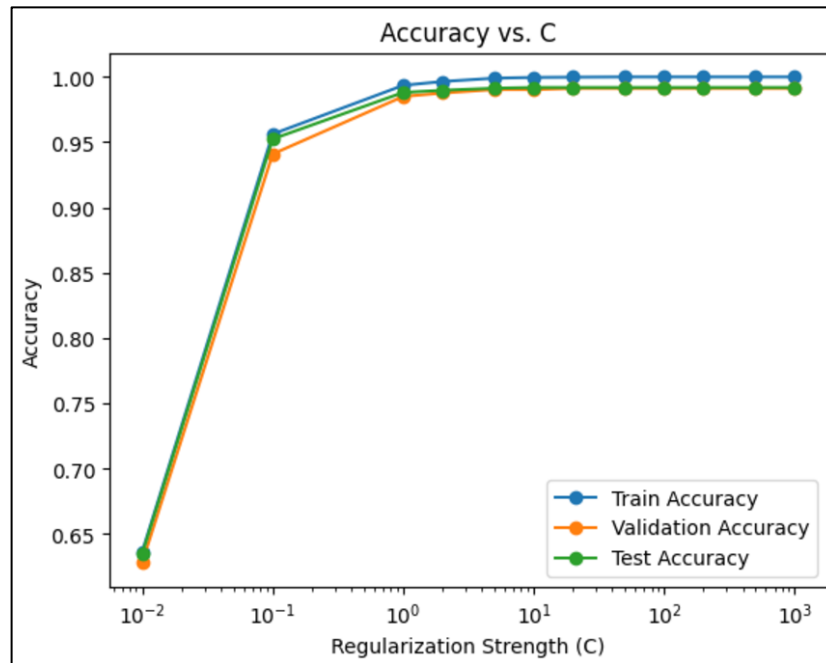


Sample Size Plot

Final Hyperparameters

SVM Hyperparameters

Parameter	Value
C	10
tol	0.001
max_iter	-1
probability	True
Kernel	poly



CIFAR10 Dataset

Binary Classes



Automobile



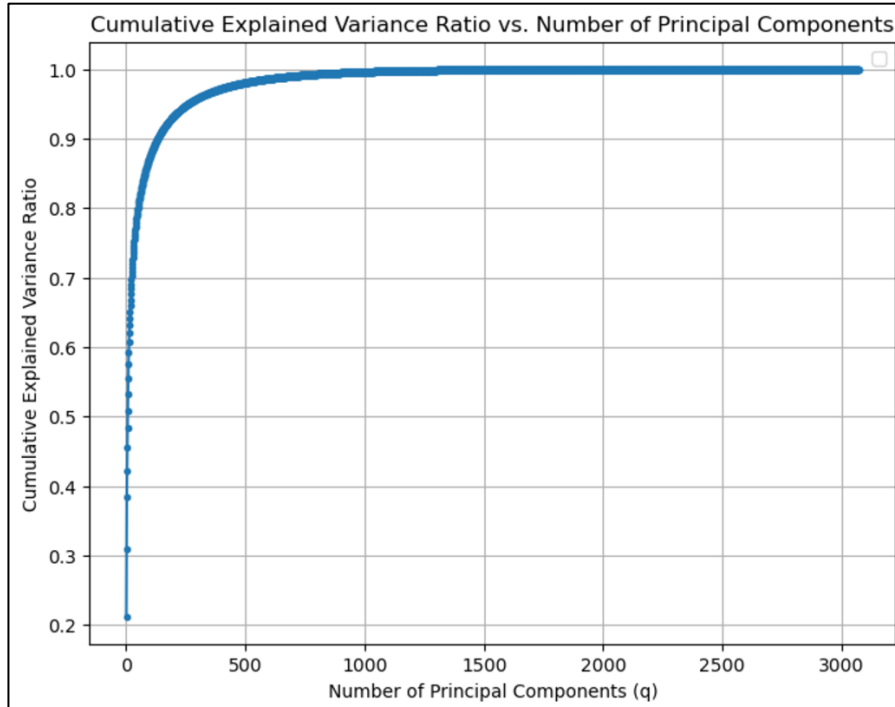
Truck

PCA + Feed-forward NN

PCA + FNN Hyperparameters

Parameter	Range	Description	Why Tuned?
n_components	[5, 200]	Number of principal components to provide as input to the model	To decide between the number of features of the dataset that we have to keep and balance the tradeoff between complexity and accuracy
Hidden layer 1 size	[4, 256]	The number of neurons in the 1 st hidden layer of the FNN	To determine the optimal capacity of the network. A larger number of neurons can potentially capture more complex patterns, but it can also lead to overfitting.
Hidden layer 2 size	[4, 256]	The number of neurons in the 3 rd hidden layer of the FNN	Similar to the previous reason, to find the optimal balance between model complexity and performance.
Hidden layer 3 size	[4, 128]	The number of neurons in the 1 st hidden layer of the FNN	Similar to the previous reason for hidden layer 1 size, to find the optimal balance between model complexity and performance.
Learning Rate	[1e-4, 1e-2]	The step size of the algorithm which determines the magnitude of the update to the model	To control the convergence speed and stability of the training process. A smaller learning rate can lead to slower convergence but more stable results, while a larger learning rate can accelerate convergence but may result in unstable training.

PCA Variance Explanation



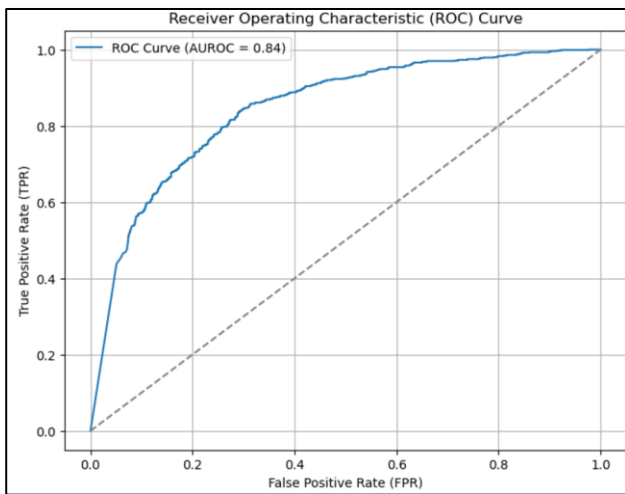
The cumulative explained variance ratio is a useful measure to understand how much information of the dataset is captured by a set of principal components. This can be very useful in determining the number of principal components we need to retain while performing dimensionality reduction.

For this dataset **259** principal components are sufficient to capture 95% of variance in the dataset if the dataset is **normalized**, otherwise **252** principal components are sufficient to capture 95% of variance in the dataset if it is not **normalized**. This number was useful in determining the range of parameters we need to optimize over. The formula for the explained variance is given below where q represents the number of components we choose to retain and d represents the total number of components.

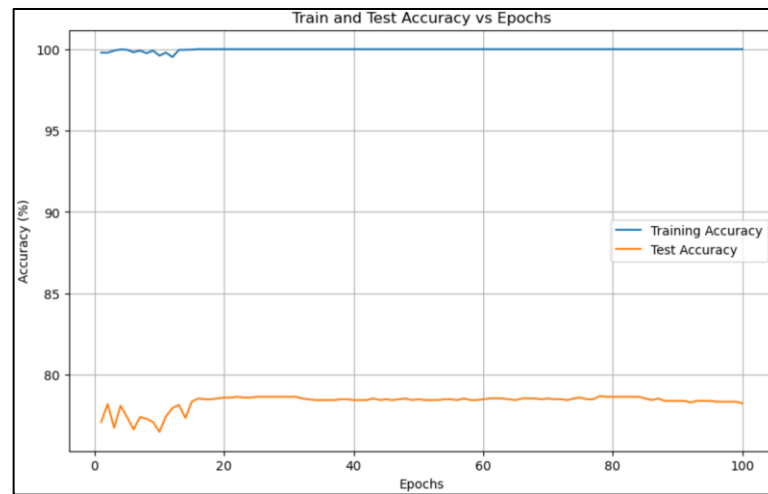
$$\sum_{i=1}^q \frac{s_i}{\sum_{j=1}^d s_j}$$

Results and Plots

Method	Accuracy	Per-class Accuracy (+1/-1)	AUROC
PCA + FNN	76.4%	Automobile: 77.00% Truck: 75.80%	0.84



ROC Plot

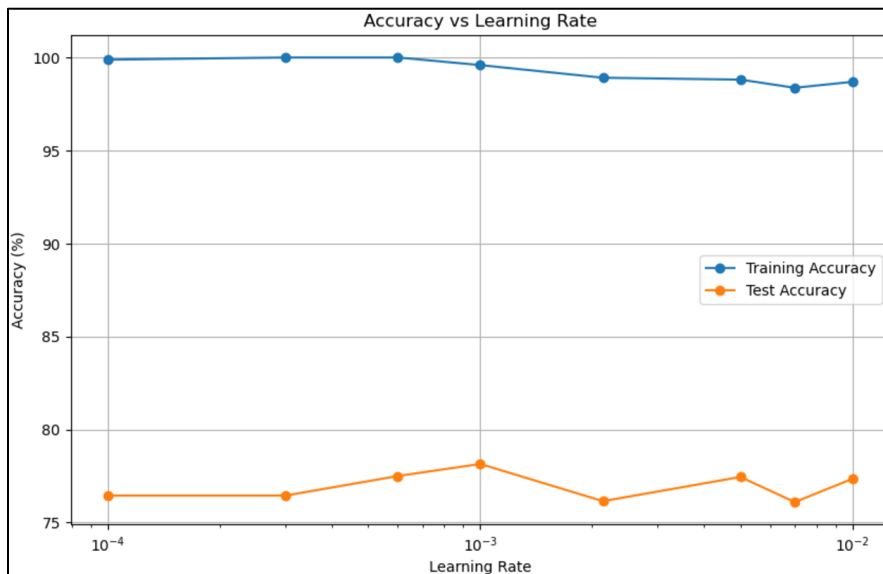


Training Epochs Plot

Final Hyperparameters

PCA + FNN Hyperparameters

Parameter	Value
n_components	112
Hidden_size1	202
Hidden_size2	165
Hidden_size3	46
Learning Rate	0.0021371291 03186701



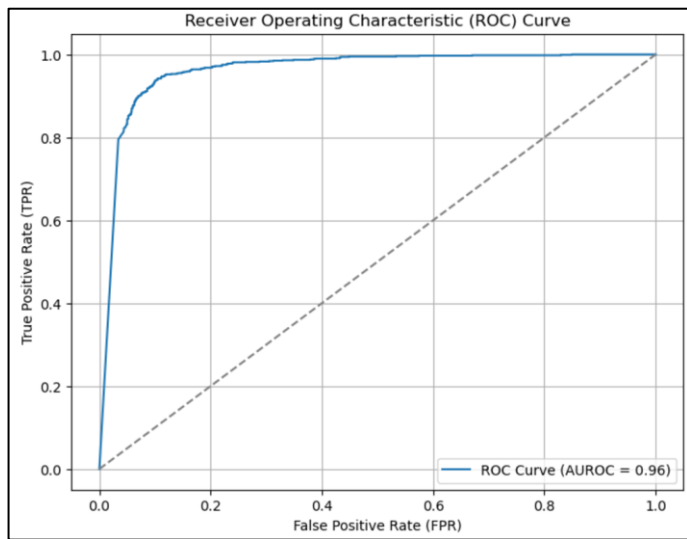
Convolutional Neural Network

CNN Hyperparameters

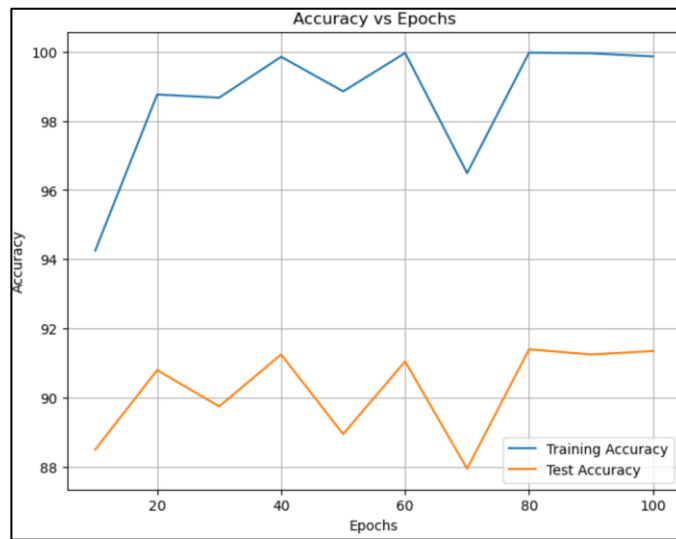
Parameter	Range	Description	Why Tuned?
Hidden layer 1 size	[4, 256]	The number of neurons in the 1 st hidden layer of the FNN	To determine the optimal capacity of the network. A larger number of neurons can potentially capture more complex patterns, but it can also lead to overfitting.
Hidden layer 2 size	[4, 256]	The number of neurons in the 2 nd hidden layer of the FNN	Similar to the previous reason, to find the optimal balance between model complexity and performance.
Learning Rate	[1e-4, 1e-2]	The step size of the algorithm which determines the magnitude of the update to the model	To control the convergence speed and stability of the training process. A smaller learning rate can lead to slower convergence but more stable results, while a larger learning rate can accelerate convergence but may result in unstable training.

Results and Plots

Method	Accuracy	Per-class Accuracy (+1/-1)	AUROC
CNN	91.45%	Automobile: 90.60% Truck: 92.30%	0.96



ROC Plot

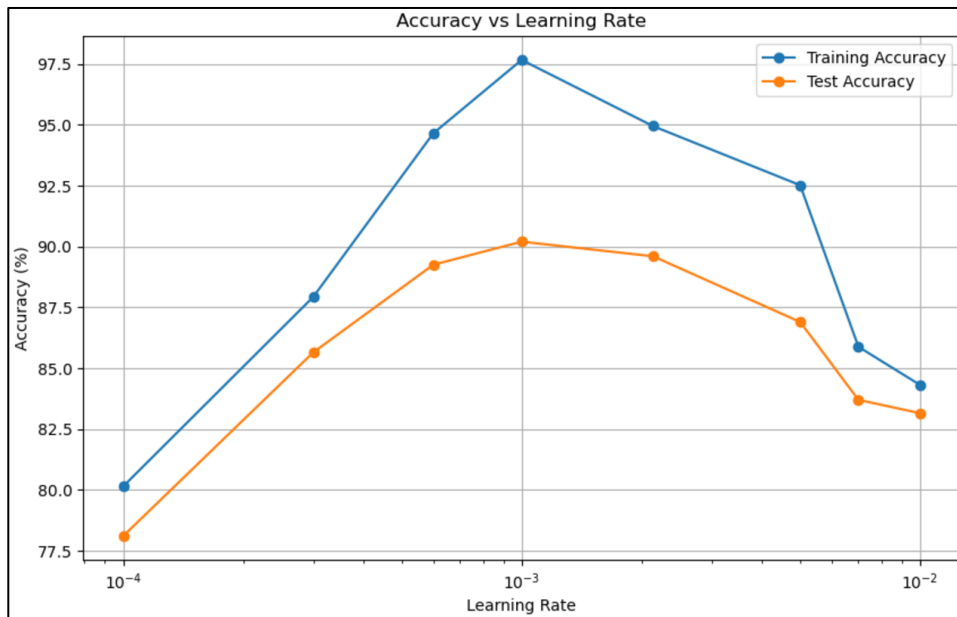


Training Epochs Plot

Final Hyperparameters

CNN Hyperparameters

Parameter	Value
Optimizer	Adam
Hidden Layer 1 Size	64
Hidden Layer 2 Size	32
Standard Kernel/Filter Size	3
MaxPool Kernel/Filter Size	2



Tuning the model over varying learning rates

Explanations

Logistic Regression With and Without Regularization

- I have modelled the without regularization scenario using a very high value for the regularization parameter C as I think that plotting it the accuracy versus any other parameter is not very insightful.
- I have made use of Random Search for optimizing the hyperparameters here

SVM

- I have made use of Random Search for optimizing the hyperparameters here

PCA + FNN

- As the problem of optimizing the structure of the FNN is particularly complex I made use of the AutoML framework optuna here. I was interested in experimenting with multiple methods for hyperparameter optimization and to that end I have used Optuna here.
- My model was able to have better performance for data that was not normalized compared to data that was normalized.

Explanations #2

CNN

- I was lucky enough to land on a good kernel size for the convolution layers. The main focus of my optimization was the FNN that was tacked on to the output of the CNN. I used Optuna to help me optimize this, but due to a shortage of computing resources I was not able to run this to completion. I had to settle with working with a standard structure for the FNN, but I do think that the test accuracy was above 90% even in this case.

Common:

- Please bear in mind while running the code that it is unlikely especially for the CNN's and FNN models to reproduce the exact same results that have been detailed in the report. Their accuracies generated tend to vary as each time the model may have slight variations in the way it learns. But by and large, the model learned will perform very close to what has been stated in the report.

References

Jennewein, Douglas M. et al. "The Sol Supercomputer at Arizona State University." In Practice and Experience in Advanced Research Computing (pp. 296–301). Association for Computing Machinery, 2023

D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Mathematical Programming*, vol. 45, no. 1, pp. 503–528, 1989, doi: <https://doi.org/10.1007/BF01589116>. Available: <https://doi.org/10.1007/BF01589116>

D. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," Computer Science, 2014, doi: <https://doi.org/10.48550/arXiv.1412.6980>.

https://scikit-learn.org/1.5/modules/generated/sklearn.linear_model.LogisticRegression.html

<https://scikit-learn.org/1.5/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC>